Classes & Objects (Basic Awareness)

Starting with a Simple Question

When you look around you, you see many things:

- A student
- A mobile phone
- A car
- A book

Each thing has:

- Some **data** (name, color, price)
- Some **actions** (study, call, drive, read)

Java tries to model real life in a similar way.

This is where **classes and objects** come in.

(If this sounds abstract now, relax. We will slowly connect it to code.)

Why Java Needs Classes and Objects

Earlier, we used variables like:

```
int age = 20;

String name = "Rahul";
```

This works fine for small programs.

But imagine storing:

- 100 students
- Each with name, age, marks

Using only variables becomes messy.

Classes help us:

- Group related data
- Represent real-world entities
- Write clean and organized programs

This is why Java is called an **object-oriented language**.

What is a Class?

A **class is a blueprint or template**.

It defines:

- What data an object will have
- What actions an object can perform

Important point:

- A class does NOT store real data
- It only defines structure

Real-life example:

- Blueprint of a house
- Design of a car

The blueprint itself is not a house.
Similarly, a class itself is not an object.

Simple Class Example

```
class Student {
    int age;
    String name;
}
```

Let us read this like a beginner:

- **class** - keyword used to create a class
- **Student** - name of the class
- **age, name** - variables inside the class

This means:

Every Student object will have an age and a name.

At this stage:

- No memory is used
- No values exist yet

(This is a very common confusion. Class does not hold real data.)

What is an Object?

An **object is a real instance of a class**.

If class is a blueprint,
object is the actual thing created from it.

Example:

- Class: Student

- Object: One student named Rahul, age 20

Objects:

- Use memory
- Store actual values
- Represent real entities

Creating an Object in Java

To create an object, Java uses the new keyword.

Example:

```
Student s1 = new Student( );
```

Let us break this clearly:

- **Student** - class name
- **s1** - reference variable
- **new** - creates object in memory
- **Student( )** - constructor call

Now:

- Memory is allocated
- Object exists
- Variables inside class are ready to use

(Pause here. This line is very important in Java.)

Accessing Data Using an Object

Once object is created, we can access its data.

Example:

```
s1.age = 20;
s1.name = "Rahul";
```

Explanation:

- **s1** - object reference
- **.** - dot operator
- **age, name** - variables inside class

Think like this:

Object dot variable

This dot operator is used everywhere in Java.

Reading Object Data

We can also read values from object.

Example:

```
System.out.println(s1.age);
System.out.println(s1.name);
```

Here:

- Object provides data
- Program prints it

(This shows object is holding real values.)

Class with Methods (Basic Idea)

Classes can contain:

- Variables (data)
- Methods (actions)

Example:

```
class Student {
   void study() {
      System.out.println("Student is studying");
   }
}
```

This means:

- Every Student object can perform study action

Methods define behavior.

Calling a Method Using Object

Methods are called using object reference.

Example:

```
s1.study();
```

Explanation:

- **s1** - object
- **.** - access operator
- **study( )** - method call

This reads like:

Student object is studying

(This is how Java maps real life to code.)

Multiple Objects from Same Class

One class can create many objects.

Example:

```
Student s1 = new Student();

Student s2 = new Student();
```

Important points:

- Both objects use same class
- Each object has separate memory
- Values do not mix

This is very useful when handling lists of data.

Common Beginner Confusions (Important)

- Class and object are not the same
- Class does not store values
- Object stores real data
- Dot operator is mandatory to access data

If these four points are clear, you are on the right path.

Where This Concept is Used

Classes and objects are used in:

- Student management systems
- Banking applications
- Employee records
- Any real-world software

Even though we keep this basic here, this idea is the foundation.

Remember This

Class defines structure.
Object stores real data.

Read this again before moving ahead.